

Theme Park · Petting Zoo — Helper Functions — English Worksheet (Beginner)

Topic: Petting Zoo — Helper Functions · **Course:** Theme Park Creator · **Level:** Beginner · **Time:** about 35 minutes

Your park has stalls and amenities. Now it needs a **petting zoo** — a fence around it, six cages inside, an animal in every cage, and somewhere to eat and drink.

Every cage is the same job. So you write that job **once**, then **call** it for every animal.

These are the tools you use this lesson:

```
corner1 = pos(1, 0, 1)
corner2 = pos(10, 0, 10)
blocks.fill(OAK_FENCE, corner1, corner2, FillOperation.REPLACE)
mobs.spawn(CHICKEN, pos(6, 0, 6))

for i in range(10):
    mobs.spawn(CHICKEN, pos(6, 0, 6))
```

`corner1` and `corner2` are the **two corners** of the box you fill. `FillOperation.REPLACE` fills the **whole solid** box. `AIR` is a block like any other — filling with `AIR` **deletes** what was there. `mobs.spawn(animal, pos(x, y, z))` puts **one** animal in **one** spot. `for i in range(10):` runs the line under it **10 times**. `OAK_FENCE` = the fence. `HAY_BLOCK` = the food. `WATER` = the drink.

These are the tools. **Turning the numbers into parameters is your job.**

1 · Solid, Then Carve

There is no "hollow box" block. So you build a box in **two moves**:

move 1 – fill it solid

```
  1  2  3  4  5  6
6  ## ## ## ## ## ##
5  ## ## ## ## ## ##
4  ## ## ## ## ## ##
3  ## ## ## ## ## ##
2  ## ## ## ## ## ##
1  ## ## ## ## ## ##
```

-->

move 2 – carve the middle out

```
  1  2  3  4  5  6
6  ## ## ## ## ## ##
5  ## . . . . ##
4  ## . . . . ##
3  ## . . . . ##
2  ## . . . . ##
1  ## ## ## ## ## ##
```

fill OAK_FENCE

(1, 1) to (6, 6)

fill AIR, one block in

(2, 2) to (5, 5)

Move 2 is one block inside move 1 on every side. The edge it leaves behind is your fence.

2 · Functions 🧩

What a helper function is

	Regular function	Helper function
Who calls it?	You do. You type it and run it.	Another function does.
How big is the job?	The whole thing.	One small job.
In your zoo	<code>build_zoo</code>	<code>fence</code> · <code>feed_area</code>
Written differently?	No — same <code>def</code> .	No — same <code>def</code> .

⚠️ There is **no special keyword** for a helper. A helper is just a function that **other functions use**. The word describes its **job**, not its code.

How to tell them apart: look at who calls it. If you type it yourself, it's the regular one. If only other functions type it, it's a helper.

Functions inside functions 🙋

A function can **call** another function. When it does, it **stops and waits** — the second function runs all the way to the end, then the first one carries on from where it stopped.

Follow the arrows. `-->` means *go into*. `<--` means *finished, come back*.

```
YOU TYPE:  build_zoo()
|
|  line 1 --> build_cage(CHICKEN, 1, 1, 10, 10)
|              |
|              |-- fill the fence box
|              |-- carve the middle out
|              |-- spawn the chicken
|              <-- finished. back to build_zoo.
|
|  line 2 --> build_cage(COW, 13, 1, 22, 10)
|              |-- same three jobs, new numbers
|              <-- finished. back to build_zoo.
|
|  line 3 --> build_cage(PIG, 25, 1, 34, 10)
|              <-- finished. back to build_zoo.
|
v  (three more, then build_zoo is done)
```

The rule: nothing is skipped. `build_zoo` cannot get to its next line until the function it called has completely finished.

Work it out step by step:

Step 1. You type `build_zoo()`. Only that. One line.

Step 2. `build_zoo` runs its **first** line. That line calls `build_cage`.

Step 3. `build_zoo` **stops and waits**. `build_cage` is running now.

Step 4. `build_cage` does its three jobs — fence, carve, animal. Then it ends.

Step 5. `build_zoo` **wakes up** and runs its **second** line. Another `build_cage`.

Step 6. Repeat until all six are done.

How to write a helper

Four steps, every time:

Step 1 — Find the job that repeats. Six cages. Same job, six times.

Step 2 — Write the job once. Give it a name that says the job.

Step 3 — Put the changing parts in the brackets. Those become **parameters**.

Step 4 — Call it, with different arguments each time.

Here is the **plan** for `build_cage`. The code is yours to write — this is only the order of the jobs:

```
def build_cage(animal, x, z, x2, z2):  
    # 1. fill a solid box of fence, corner to corner  
    # 2. carve the middle back to AIR, one block in  
    # 3. put the animal inside
```

3 · Predict 🧠

Read the code. Circle your answer.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)
```

What is on the ground now? Circle one: a solid square of fence · a fence ring · nothing

```
blocks.fill(AIR, pos(2, 0, 2), pos(9, 0, 9), FillOperation.REPLACE)
```

Now what is left? Circle one: a fence ring · a solid square · nothing

The animal has to stand inside. Which move makes room for it? Circle one: move 1 · move 2

In `build_cage(SHEEP, 1, 13, 10, 22)`, what goes into the box named `animal`? Circle one:

`SHEEP` · `1` · `13`

4 · Spot the Bug 🐛

Bug A — This should leave a fence ring with a space inside.

```
blocks.fill(OAK_FENCE, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)  
blocks.fill(AIR, pos(1, 0, 1), pos(10, 0, 10),  
            FillOperation.REPLACE)
```

What is left on the ground? Circle one: nothing · a fence ring · a solid square

Why does that happen?

Bug B — Six animals should stand in six different cages.

```
build_cage(CHICKEN, 1, 1, 10, 10)  
build_cage(COW, 1, 1, 10, 10)  
build_cage(PIG, 1, 1, 10, 10)
```

What is wrong? Circle one: every cage is in the same spot · the animals are the same · the helper is missing

How do you know?

Switch to your homework world. **Work through these in order — do not skip to step 6.**

Step 1. Write the line `def build_cage(animal, x, z, x2, z2):`

Step 2. Inside it, make `corner1` and `corner2` out of the **parameters** — not out of fixed numbers.

Step 3. Fill that box solid with `OAK_FENCE`.

Step 4. Fill **AIR** **one block inside** the box. Work out what to add to **x** and **z**, and what to take off **x2** and **z2**.

Step 5. Spawn the animal **in the middle**, not on the corner.

Step 6. Call it **once**, with **CHICKEN** . Run it. Look at it. **Fix it before you go on.**

Step 7. Now write the other five calls, with five different animals.

Step 8. Put one big `OAK_FENCE` box around the whole zoo.

Animals: CHICKEN · COW · PIG · SHEEP · RABBIT · LLAMA

Cage corners that fit — front row and back row:

z 13-22	[SHEEP]	[RABBIT]	[LLAMA]
z 1-10	[CHICKEN]	[COW]	[PIG]
	x 1-10	x 13-22	x 25-34

Write your helper and your six calls:

Your helper is written once but runs six times. Explain how that works.

6 · Show Your Work 📷 🎥

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1 · Your code. Scroll through it. Say what each part does.

2 · Your build. Point the camera. Name the parts.

Say these out loud, filling in the blanks:

Today I built ____.

I built it using this code: ____.

In this code I used ____.

The hardest part was ____.

That part was hard because ____.

The most fun part was ____.

Something new I learned was ____.

Submit ✅

Send this worksheet + your video to teacher on KakaoTalk.